



FSBM PLATFORM

Guide de dépannage

Crash & Processus zombies sur les ports

Manuel pratique de diagnostic Windows

Ce guide vous aide à résoudre tous les cas de blocage de ports rencontrés lors du développement de la plateforme FSBM sous Windows. Inclut diagnostic, solutions rapides et fix définitif.

Équipe PFE — 2025/2026

AKRAM BELMOUSSA · ZAKARIA · NOUHAILA

Sommaire

Chapitre 1 — Identifier un processus zombie	3
1.1 Symptômes typiques	3
1.2 Tableau de diagnostic rapide	4
1.3 Commandes de détection	5
Chapitre 2 — Pourquoi ça arrive ?	6
2.1 WSL2 et les proxies de port	6
2.2 Hyper-V et la plage dynamique	7
2.3 TIME_WAIT et sockets bloqués	8
Chapitre 3 — Solution RAPIDE	9
3.1 Le script clean-zombies.ps1	9
3.2 Méthodes manuelles	10
Chapitre 4 — Solution PERMANENTE	12
4.1 Lancer fix-windows-ports.ps1 (admin)	12
4.2 Vérification	13
4.3 Bonnes pratiques	14
Chapitre 5 — Cas particuliers	15
5.1 WinError 10013	15
5.2 Port reste en LISTEN après pkill	16
5.3 Plusieurs services qui se battent	17
Annexes	18
A — Scripts complets	18
B — Glossaire technique	20
C — Commandes de référence	21

Chapitre 1 — Identifier un processus zombie

1.1 Symptômes typiques

Un **processus zombie** dans le contexte des ports réseau désigne un état dans lequel un port est occupé par un processus qui semble mort ou inaccessible. Voici les signes qui doivent vous alerter :

- Le serveur (uvicorn, npm, etc.) refuse de démarrer avec un message d'erreur lié à un port.
- **WinError 10013** : « Une tentative d'accès à un socket de manière interdite par ses autorisations d'accès a été tentée. »
- **WinError 10048** : « Une seule utilisation de chaque adresse de socket est autorisée. »
- Message « *Address already in use* » alors que vous venez de fermer le terminal.
- `taskkill /F /PID xxx` répond « *processus introuvable* » alors que `netstat` montre que le PID écoute.
- Le navigateur charge encore l'ancienne version de l'application alors que vous l'avez modifiée.
- Le port reste en état LISTEN alors qu'aucun processus visible ne devrait l'utiliser.

[!] Diagnostic immédiat

Si vous voyez un de ces symptômes, vous êtes face à un **processus zombie** ou à une **réservation de port** par Windows. Lisez la suite pour comprendre et résoudre.

1.2 Tableau de diagnostic rapide

Symptôme	Cause probable	Solution
WinError 10013	Port réservé par Hyper-V ou Windows (plage dynamique)	Lancer <code>fix-windows-ports.ps1</code> en admin pour réserver le port
WinError 10048	Un autre processus écoute déjà sur le port	Identifier et tuer le processus avec <code>clean-zombies.ps1</code>
<code>taskkill</code> : processus introuvable	Le proxy de port WSL/Hyper-V persiste sans processus visible	<code>wsl --shutdown</code> + <code>Restart-Service hns</code>
Port en TIME_WAIT	Socket fermé récemment, attente du timeout réseau (4 min)	Attendre ou changer de port
Plusieurs PID sur le même port	Plusieurs instances ont été lancées (npm start dupliqué)	Kill tous les <code>python.exe</code> / <code>node.exe</code> liés
Le port se libère puis se reprend	Auto-reload Angular/Vite réutilise le PID	Couper Ctrl+C dans la fenêtre d'origine

1.3 Commandes de détection

Avant toute action, identifiez précisément l'état du port avec ces commandes :

Voir qui utilise un port précis (ex 5002)

```
# PowerShell natif (recommandé)
Get-NetTCPConnection -LocalPort 5002 | Format-List

# CMD classique
netstat -ano | findstr :5002

# Identifier le processus à partir du PID
Get-Process -Id 17968
```

Lister tous les ports écoutés

```
Get-NetTCPConnection -State Listen |
    Where-Object { $_.LocalAddress -in @('127.0.0.1', ':::1', '0.0.0.0') } |
    Select-Object LocalPort, OwningProcess,
        @{N='Process';E={(Get-Process -Id $_.OwningProcess -EA SilentlyContinue).ProcessName}} |
    Sort-Object LocalPort
```

Voir les plages réservées par Hyner-V

```
netsh int ipv4 show excludedportrange protocol=tcp

# Exemple de sortie :
#      5357          5357
#      50000         50059      *
#      * = exclusion administrative
```

Chapitre 2 — Pourquoi ça arrive ?

2.1 WSL2 et les proxies de port

WSL2 (Windows Subsystem for Linux v2) tourne dans une **machine virtuelle légère** isolée du système hôte. Pour que les serveurs lancés dans WSL soient accessibles depuis Windows (sur localhost), Windows installe automatiquement un **proxy de port** qui redirige les connexions du host vers la VM.

Le problème : Ce proxy peut survivre à la mort du processus WSL qui l'avait créé. Résultat : Windows voit qu'un PID écoute sur le port, mais le processus n'existe plus dans WSL — donc taskkill ne peut rien faire.

[i] C'est exactement ce qui s'est passé pendant le développement de FSBM Platform. Les tests automatisés lancés depuis Bash (WSL) ont créé des proxies persistants sur les ports 5001 et 5002, qui sont restés actifs jusqu'à redémarrage du service réseau.

Comment libérer un proxy WSL :

```
# Tuer toutes les instances WSL et libérer les proxies
wsl --shutdown

# Restart du service réseau Windows (admin recommandé)
Restart-Service hns -Force

# Si même hns ne suffit pas (cas extrême)
Stop-Service vmcompute -Force
Start-Service vmcompute
```

2.2 Hyper-V et la plage dynamique

Windows réserve une plage de ports pour ses besoins internes et pour Hyper-V (virtualisation). Cette plage, appelée **plage dynamique**, peut grimper haut et inclure des ports « normalement libres » comme 5000-5100.

Quand vous essayez d'écouter sur un port qui tombe dans cette plage réservée, Windows refuse avec **WinError 10013**.

La solution : ajouter une *exclusion de port* qui force Windows à **ne jamais** allouer dynamiquement ce port. Le port devient ainsi réservé exclusivement pour vous.

```
# (admin requis)
netsh int ipv4 add excludedportrange protocol=tcp startport=5002 numberofports=1

# Vérifier l'exclusion
netsh int ipv4 show excludedportrange protocol=tcp
```

2.3 TIME_WAIT et sockets bloqués

Lorsqu'un serveur ferme une connexion TCP, le socket reste en état TIME_WAIT pendant **2 à 4 minutes** (selon la config). Cela permet au système d'éviter de réutiliser le même port pour une connexion radicalement différente.

Si vous relancez immédiatement un serveur sur le même port, il peut échouer avec « **Address already in use** ». Solutions :

- **Attendre 4 minutes** et réessayer (le TIME_WAIT se libère naturellement).
- **Changer de port** temporairement (ex. 8002 au lieu de 5002).
- **Activer SO_REUSEADDR** sur le serveur (déjà actif par défaut dans uvicorn).
- **Vérifier l'état** du socket avec Get-NetTCPConnection — si l'état est TIME_WAIT, c'est normal.

Chapitre 3 — Solution RAPIDE

3.1 Le script clean-zombies.ps1

Le script `scripts\clean-zombies.ps1` automatise les **5 méthodes** de nettoyage en cascade. Il fonctionne **sans privilèges administrateur** dans la plupart des cas.

Lancement

```
cd C:\Users\belmo\studies\PFE\chatbot-fsbm-platform
powershell -ExecutionPolicy Bypass -File .\scripts\clean-zombies.ps1
```

Ce que fait le script

Étape 1 — Stop-Process. Tentative de tuer chaque PID identifié comme occupant un port FSBM via `Stop-Process -Force`.

Étape 2 — taskkill /F /T. Plus brutal : force-kill du processus ET de tout son arbre de processus enfants (utile si npm a démarré plusieurs node).

Étape 3 — Recherche par CommandLine. Liste TOUS les `python.exe` / `node.exe` du système et tue ceux dont la ligne de commande contient « `uvicorn` », « `chatbot-fsbm` » ou « `ng serve` ».

Étape 4 — wsl --shutdown. Libère les proxies de port créés par WSL (la cause #1 des zombies persistants).

Étape 5 — Restart hns. Redémarre le Host Network Service de Windows, qui gère les ports proxy. Nécessite l'admin pour fonctionner pleinement.

[OK] Le script réussit dans **90% des cas** sans admin. Pour les cas extrêmes (blocage Hyper-V), passez au Chapitre 4.

3.2 Méthodes manuelles

Si vous préférez agir manuellement (debugging, apprentissage), voici les commandes à exécuter dans l'ordre :

Étape A — Identifier le coupable

```
# Lister tous les PID sur les ports FSBM
@(5001, 5002, 4200, 8001, 8002) | ForEach-Object {
    $c = Get-NetTCPConnection -LocalPort $_ -EA SilentlyContinue
    if ($c) { Write-Host "Port $_ -> PID $($c.OwningProcess)" }
}
```

Étape B — Tuer en force

```
# Pour chaque PID identifié
taskkill /F /T /PID 17968

# Si le PID n'existe plus mais le port reste occupé
wsl --shutdown
Restart-Service hns -Force # admin requis
```

Étape C — Vérification

```
Get-NetTCPConnection -LocalPort 5002 -EA SilentlyContinue  
# Aucune sortie = port libre, vous pouvez relancer
```

[!] Si après toutes ces étapes le port reste bloqué, c'est probablement une **exclusion Hyper-V**.
Passez à la solution permanente du Chapitre 4.

Chapitre 4 — Solution PERMANENTE

4.1 Lancer fix-windows-ports.ps1 (admin)

Cette solution est **définitive** : une fois appliquée, les ports FSBM sont réservés à votre usage exclusif. Aucun service Windows ne pourra plus les voler accidentellement.

[!] Prérequis

Ce script **nécessite les droits administrateur** car il modifie la configuration réseau du système.

Étape 1 — Ouvrir PowerShell en administrateur

Plusieurs méthodes :

- Touche **Windows** → tape « PowerShell » → clic droit → « Exécuter en tant qu'administrateur »
- Touche **Windows + X** → « Terminal Windows (admin) »
- Dans l'Explorateur de fichiers, **Shift + clic droit** dans un dossier vide → « Ouvrir dans Terminal »

Étape 2 — Lancer le script

```
cd C:\Users\belmo\studies\PFE\chatbot-fsbm-platform
powershell -ExecutionPolicy Bypass -File .\scripts\fix-windows-ports.ps1
```

Étape 3 — Ce que fait le script

1. **Vérification admin** — Refuse de continuer s'il n'est pas en admin (sécurité).
2. **Liste des plages réservées** — Affiche les exclusions Hyper-V actuelles via netsh.
3. **Kill des processus actuels** — taskkill /F /T sur tous les PID sur les ports FSBM.
4. **Arrêt des services réseau** — Stop WinNat, hns, vmcompute pour libérer les locks au niveau Windows.
5. **Ajout des exclusions** — netsh int ipv4 add excludedportrange pour 5001, 5002, 8001, 8002, 4200.
6. **Redémarrage des services** — Restart de WinNat, hns, vmcompute.
7. **WSL shutdown** — Libère les proxies WSL résiduels.
8. **Vérification finale** — Confirme que tous les ports sont libres.

4.2 Vérification

Après le script, vous devriez voir les ports FSBM listés dans les exclusions :

```
netsh int ipv4 show excludedportrange protocol=tcp

# Sortie attendue :
# Port de début   Port de fin
```

```
# ██████████ ██████████
#      4200          4200    *
#      5001          5001    *
#      5002          5002    *
#      8001          8001    *
#      8002          8002    *
#      (* = exclusion administrative)
```

[OK] Une fois ces exclusions en place, ces ports sont **garantis libres** au démarrage de vos services. Hyper-V, Docker, WSL ne pourront plus jamais les voler.

4.3 Bonnes pratiques pour ne plus jamais avoir le problème

Lancer les serveurs depuis Windows natif — Préférez cmd.exe ou PowerShell Windows. Évitez WSL/bash pour les serveurs persistants.

Arrêter proprement — Utilisez toujours Ctrl+C dans la fenêtre où le serveur tourne. Ne fermez pas brutalement la fenêtre.

Utiliser des ports élevés — Pour des tests rapides, préférez 8001-8999 (moins de conflits avec Hyper-V).

Vérifier avant de lancer — Lancez clean-zombies.ps1 si vous avez des doutes.

Nettoyage régulier — Une fois par semaine, lancez `wsl --shutdown` si vous utilisez WSL.

Réservez vos ports — `fix-windows-ports.ps1` est à faire **une fois** au début du projet. Ensuite, tout fonctionne.

Chapitre 5 — Cas particuliers

5.1 WinError 10013

[X] WinError 10013 : Une tentative d'accès à un socket de manière interdite par ses autorisations d'accès a été tentée.

Cause : Le port que vous essayez d'écouter est *réservé* par Hyper-V / WSL / un service Windows. Windows refuse de vous le donner.

Solution :

```
# Vérifier si le port est réservé
netsh int ipv4 show excludedportrange protocol=tcp | findstr 5002

# Si oui, lancer le fix permanent en admin
.\scripts\fix-windows-ports.ps1

# Solution alternative : utiliser un autre port
py -m uvicorn app.main:app --port 8002
```

5.2 Port reste en LISTEN après kill

[!] Symptôme : Vous avez tué le processus, mais Get-NetTCPConnection -LocalPort X montre toujours un PID en état LISTEN — qui n'existe plus.

Cause : Le PID était un proxy WSL, pas un vrai processus Windows. Windows ne peut pas le libérer sans intervention.

Solution :

```
wsl --shutdown
Start-Sleep 3
Restart-Service hns -Force # admin requis
Start-Sleep 5
Get-NetTCPConnection -LocalPort 5002 # devrait être vide
```

5.3 Plusieurs services qui se battent

[!] Symptôme : Vous voyez plusieurs PID python.exe sur le même port, le serveur démarre puis se ferme tout seul.

Cause : Vous avez lancé plusieurs instances de uvicorn par erreur (par exemple en cliquant deux fois sur SETUP.bat).

Solution :

```
# Tuer TOUS les python.exe liés à uvicorn
Get-Process python -EA SilentlyContinue | ForEach-Object {
    $cmd = (Get-CimInstance Win32_Process -Filter "ProcessId=$( $_.Id )").CommandLine
    if ($cmd -match 'uvicorn') {
        Write-Host "Killing PID $( $_.Id )"
    }
}
```

```
        Stop-Process -Id $_.Id -Force
    }
}

# Relancer une seule instance
py -m uvicorn app.main:app --port 5002
```

Annexes

Annexe A — Scripts complets fournis

Trois scripts sont fournis dans `chatbot-fsbm-platform/scripts/` :

Script	Rôle	Admin ?
<code>clean-zombies.ps1</code>	Nettoyage rapide en 5 étapes (Stop-Process → taskkill → CommandLine → WSL → hns).	Non
<code>fix-windows-ports.ps1</code>	Fix DEFINITIF : réserve les ports FSBM via netsh exclusion. À faire UNE FOIS.	OUI
<code>start.ps1</code>	Lance les 3 services (academic, chatbot, frontend) dans 3 fenêtres séparées.	Non

Annexe B — Glossaire technique

Zombie. Processus apparemment mort mais dont le socket est toujours alloué côté kernel.

Port. Numéro entre 1 et 65535 identifiant une application qui écoute sur le réseau.

LISTEN. État d'un socket qui attend une connexion entrante.

TIME_WAIT. État transitoire d'un socket fermé, dure 2-4 min, empêche la réutilisation immédiate.

Hyper-V. Hyperviseur de Microsoft, utilisé par Docker Desktop, WSL2, sandbox Windows.

WSL2. Windows Subsystem for Linux v2 — VM Linux légère intégrée à Windows.

hns. Host Network Service — gère les réseaux virtuels Hyper-V/Docker/WSL.

WinNat. Windows NAT Driver — réalise la translation d'adresses entre WSL/Docker et l'hôte.

netsh. Outil en ligne de commande Windows pour configurer le réseau (firewall, ports, NAT).

Excluded Port Range. Plage de ports qu'on demande à Windows de NE PAS allouer dynamiquement.

Annexe C — Commandes de référence

Détection

```
Get-NetTCPConnection -LocalPort 5002
netstat -ano | findstr :5002
Get-Process -Id 17968
```

Nettoyage

```
Stop-Process -Id 17968 -Force
taskkill /F /T /PID 17968
wsl --shutdown
```

Reset réseau (admin)

```
Restart-Service hns -Force
Restart-Service WinNat -Force
Restart-Service vmcompute -Force
```

Réservation permanente (admin)

```
netsh int ipv4 add excludedportrange protocol=tcp startport=5002 numberofports=1
netsh int ipv4 delete excludedportrange protocol=tcp startport=5002 numberofports=1
netsh int ipv4 show excludedportrange protocol=tcp
```

Solution radicale

```
# Redémarrer Windows = tout est nettoyé
shutdown /r /t 0
```

[i] Ce guide complète **FSBM_Platform_Guide_Technique.pdf** (installation) et **FSBM_Platform_Rapport_PFE.pdf** (présentation du projet).